furqan-lint vs. bandit: A Finding-Count Comparison Across 25 Python Codebases

Bilal Syed Arfeen
BayyinahEnterprise

November 2026

Abstract

We compare furqan-lint v2.1.0, a static analysis tool checking for structural honesty (whether a program's surface behavior matches its substrate behavior), against bandit 1.9.4 across 25 open-source Python packages (2,327,868 lines, 5,252 files). With symmetric test-path exclusion for both tools, furqan-lint produces more findings than bandit in all 25 packages (pooled ratio 6.3x, median per-package ratio 5.3x). Without the exclusion (as run), furqan-lint leads in 22 of 25 packages (pooled 0.83x, median 5.0x), with the reversal in three scientific computing packages driven entirely by bandit's B101 (assert_used) firing on test files that furqan-lint's SI-006 exempts. Two furqan-lint rules -- SI-001 (Any annotation, 47.1%) and SI-006 (assert, 11.5%) -- account for 58.6% of output; SI-001 duplicates ruff's ANN401. Whether these counts represent a complementary analysis class requires an overlap decomposition and precision measurement we have not performed. We provide raw JSON from both tools for all 25 packages.

Note on terminology. Finding means a pattern flagged by a tool's rule. The false positive rate is not measured. We present count differences, not defect counts.

---

1. Introduction

Static analysis tools for Python fall into established categories: style checkers (ruff, flake8), type checkers (mypy, pyright), and security scanners (bandit, semgrep). We examine whether a tool organized around *structural honesty* -- gaps between what code's interface claims and what its implementation does -- produces meaningfully different findings.

furqan-lint v2.1.0 implements 42 Python rules in four families:

- **RING-CLOSE (RC)**: unclosed resources (5 rules)
- **MARAD (MR)**: silent failure (5 rules: MR-001/002/003/005/007)
- **TRUST-BOUNDARY (TB)**: security boundaries (22 active + 3 informational, off by default)
- **SCAN-INCOMPLETE (SI)**: abandoned analysis (7 rules)

Three TB rules (001/002/003) flag language features rather than likely problems and are off by default. All results use the default: 39 active rules. Eight additional rules exist only in non-Python adapters (C, C++, Java, C#, Ada/SPARK), for 50 unique rule IDs across six languages. Only the 39 active Python rules are evaluated here.

Some of furqan-lint's rules overlap with rules in existing tools. SI-001 (Any annotation) duplicates ruff's ANN401. Several TB rules (hardcoded secrets, insecure hash, eval/exec) overlap with bandit rules. Of the three motivating

patterns below, two have existing coverage: broad except-catching (ruff BLE001, pylint W0718) and hardcoded passwords (bandit B105). The third -- kwargs passthrough without validation -- appears unique to furqan-lint.

Examples of structural honesty findings:

- An `except Exception` block that catches everything but re-raises nothing.
- A parameter named `password` assigned the literal string `"admin123"`.
- `**kwargs` accepted and passed through without validation.

---

2. Method

2.1 Package selection

25 packages from PyPI, selected for domain diversity (web frameworks, HTTP clients, data science, ORMs, task queues, templating, cryptography, SSH, scraping, CLIs, cloud SDKs, image processing). Size: 6,385 lines (requests) to 635,499 (pandas). The selection is convenience-based: widely-used production packages chosen before scanning.

2.2 Tools

Tool	Version	Configuration
furqan-lint	v2.1.0	Default (39 active Python rules)
bandit	1.9.4	Default (all rules)

2.3 Procedure

Each package scanned in installed form (site-packages). Both tools run with default configuration.

Test-path asymmetry. furqan-lint's SI-006 (assert usage) exempts test files. The exact predicate: a file is considered a test if its basename starts with `test_`, ends with `_test.py`, or is `conftest.py`, OR if any directory component in its path is exactly `tests` or `test` (component matching, not substring -- `contests/` does not match). bandit's B101 (assert_used) does not exempt test files. Installed packages include test files (~36% of total lines). This asymmetry drives the 0.2x--0.3x ratios in numpy, pandas, and scipy.

Non-Python findings. furqan-lint's tree-sitter adapters scanned 76 non-Python files (.c/.h/.cpp) bundled in numpy's site-packages, producing 76 findings. These are reported separately.

2.4 Threats to validity

1. **Overlap.** We do not decompose findings into overlapping and unique categories. 2. **Precision.** Neither tool's false positive rate is measured. SI-001 (47.1% of output) may have a high false-positive rate. 3. **Test-path asymmetry.** Documented above; a sensitivity analysis follows. 4. **Rule concentration.** Two rules produce 58.6% of output. 5. **SI-001 duplicates ruff ANN401.** 6. **Single bandit version (1.9.4).** 7. **Convenience sample.** Package selection cannot be verified as unbiased.

---

3. Results

3.1 Sensitivity: as run vs. symmetric test-path exclusion

Condition	furqan total	bandit total	Pooled ratio	furqan > bandit	Median per-pkg
As run (includes 76 non-Python)	24,933	29,959	0.83x	22 / 25	5.0x
Both exclude test paths	22,888	3,615	6.33x	25 / 25	5.3x

The as-run pooled ratio (0.83x) is an artifact of the SI-006/B101 asymmetry. Under symmetric test-path exclusion, furqan-lint produces more findings in all 25 packages.

3.2 Per-package results (as run)

Package	Domain	Files	Lines	furqan	bandit	Ratio
django	Web framework	909	165,634	1,605	281	5.7x
flask	Micro web	24	9,199	76	13	5.8x
fastapi	Async web	52	22,748	688	70	9.8x
tornado	Async networking	76	50,586	947	285	3.3x
starlette	ASGI	36	6,947	313	73	4.3x
requests	HTTP library	19	6,385	149	16	9.3x
httpx	Async HTTP	23	8,828	84	29	2.9x
urllib3	HTTP low-level	36	11,914	99	10	9.9x
aiohttp	Async HTTP	54	28,467	662	178	3.7x
sqlalchemy	SQL ORM	258	253,997	8,496	866	9.8x
celery	Task queue	161	47,496	703	49	14.3x
jinja2	Template engine	25	14,425	239	62	3.9x
pydantic	Data validation	105	45,762	2,099	86	24.4x
cryptography	Crypto library	78	13,868	52	18	2.9x
paramiko	SSH library	42	18,621	204	21	9.7x
scrapy	Web scraping	199	38,503	1,361	241	5.6x
click	CLI framework	17	12,674	94	36	2.6x
rich	Rich terminal	100	38,515	200	58	3.4x
typer	CLI builder	32	13,911	486	60	8.1x
boto3	AWS SDK	39	8,558	38	0	--
PIL	Image processing	98	35,149	652	288	2.3x
werkzeug	WSGI utilities	52	21,687	184	28	6.6x
numpy	Numerical	407	243,710	1,018	3,597	0.3x
pandas	Data analysis	1,422	635,499	2,657	17,701	0.2x
scipy	Scientific	988	574,785	1,827	5,893	0.3x

3.3 Ratio distribution (as run, n=24 excluding boto3)

Metric	Value
Mean	6.2x
Median	5.0x
SD	5.3
Min	0.2x (pandas)
Max	24.4x (pydantic)

3.4 Finding density

Metric	furqan	bandit
Pooled (per KLOC)	10.7	12.9
Range (per KLOC)	3.2 -- 45.9	0.0 -- 27.8

3.5 Severity distribution (Python findings only: 24,857)

Severity	Count	Percentage
CRITICAL	297	1.2%
HIGH	1,918	7.7%
MEDIUM	17,800	71.6%
LOW	4,842	19.5%

76 non-Python findings (from numpy's bundled C code) are excluded from this table: TB-029 (34), TB-030 (19), SI-004 (12), TB-033 (4), TB-032 (4), TB-026 (2), TB-018 (1). CRITICAL Python findings: TB-006 pickle/yaml (153), TB-005 eval/exec (115), MR-001 bare except (29).

3.6 Per-rule concentration

Rule	Count	%	Description
SI-001	11,742	47.1%	Any type annotation (cf. ruff ANN401)
SI-006	2,872	11.5%	assert in non-test code (cf. bandit B101)
SI-005	2,794	11.2%	**kwargs passthrough
SI-007	2,017	8.1%	hasattr() swallows AttributeError
SI-004	1,894	7.6%	TODO/FIXME comment
MR-005	964	3.9%	Empty except block
Other (35 rules)	2,650	10.6%	

Two rules produce 58.6%. Top 6 produce 89.4%. 35 rules share the remaining 10.6%.

---

4. Overlap Decomposition

4.1 Method

We matched findings by file path and line number (window of 2 lines). Each pair is classified as *semantic* (the two rules detect related patterns, per a predefined mapping) or *coincidental* (co-located but unrelated). Bandit matching is symmetric: a bandit finding is marked overlapping if any furqan finding is within its window.

4.2 Aggregate overlap

Category	Count	% of furqan
Unique to furqan	20,803	83.4%
Overlap (semantic)	5,151	20.7%
Overlap (coincidental)	1,044	4.2%
Unique to bandit	25,913	--

83.4% of furqan-lint's findings have no bandit finding within 2 lines on the same file. Of the 16.6% that co-locate, 83.2% are semantic (related rules) and 16.8% are coincidental.

4.3 Test-path stratum

Stratum	Unique furqan	Unique bandit
Non-test paths	19,321	249
Test paths	1,482	25,664

99.0% of unique-to-bandit findings are on test paths (B101 assert_used). Excluding test paths, bandit has only 249 unique findings, primarily import warnings (B403, B404), subprocess (B603), and hardcoded secrets (B105/B106).

4.4 Dominant overlap pair

SI-006 x B101 accounts for 4,448 of 5,151 semantic overlaps (86.3%). Both rules detect assert usage; the overlap is expected and documented.

---

5. Discussion

5.1 What the decomposition shows

furqan-lint and bandit mostly flag different locations: 83.4% of furqan's findings have no bandit counterpart at the same location. This is robust across line windows (83.0% at window=0, 83.4% at 2, 81.8% at 5).

However, "unique against bandit" does not mean "unique against all tools." By rule identity, an estimated 75.9% of unique-to-furqan findings come from rules with counterparts in other linters: SI-001/ruff ANN401, SI-004/ruff TD, SI-002 and SI-003/ruff PGH, MR-002/ruff BLE001. A three-way decomposition against ruff is needed to quantify this.

The remaining ~4,880 findings with no apparent counterpart in bandit or ruff come primarily from SI-005 (kwargs passthrough, 2,784) and SI-007 (hasattr masking, 1,844).

5.2 What the decomposition does not show

Whether any of these findings -- unique or overlapping -- represent real problems. The false positive rate and safety relevance of each rule are not measured in this study. A precision study with blind manual classification is the necessary next step.

5.3 Limitations

1. Finding count is not defect count. Precision is not measured. 2. Overlap is against bandit only. ~75.9% of unique-to-furqan findings may overlap with ruff by rule identity (not verified by running ruff). 3. Two rules produce 58.6% of output. 4. SI-001 duplicates ruff ANN401. 5. Test-path asymmetry between SI-006 and B101 (sensitivity table and test-path stratum provided). 6. 76 non-Python findings from numpy's C code (reported separately). 7. Convenience sample of packages. 8. Single bandit version (1.9.4). 9. Coincidental co-location (4.2%) inflates the semantic overlap count.

---

6. Conclusion

furqan-lint v2.1.0 and bandit 1.9.4 flag different locations: 83.4% of furqan-lint's findings have no bandit finding within 2 lines. Under symmetric test-path exclusion, furqan-lint leads in all 25 packages (median 5.3x, pooled 6.3x). As run, it leads in 22 of 25 (median 5.0x, pooled 0.83x).

Two rules account for 58.6% of furqan-lint's output, one of which (SI-001) duplicates ruff ANN401. An estimated 75.9% of unique-to-furqan findings have counterparts in other linters by rule identity (not verified). The ~4,880 findings from rules with no apparent counterpart (kwargs passthrough, hasattr masking) represent the strongest candidate for genuinely novel detection.

Whether these findings represent a complementary analysis class requires a precision study and a three-way decomposition against ruff, neither of which is performed here. The raw data to answer both questions is shipped.

The tool is open-source under Apache-2.0: **https://github.com/BayyinahEnterprise/Kun-Fayakun**

Data availability

benchmark_v2.1.0.csv, raw_json/ (furqan-lint), bandit_json/ (bandit), overlap/ (decomposition CSVs and summary), benchmark.py (with --from-json replay), versions.json, rule_manifest.json, test_output_v2.1.0.txt (208 tests). Tools: furqan-lint v2.1.0, bandit 1.9.4, Python 3.13.18.

References

[1] B. S. Arfeen, "Structural Honesty Verification: On Surface-Substrate Divergence Across Systems," BayyinahEnterprise, 2026. https://github.com/BayyinahEnterprise/Kun-Fayakun

[2] PyCQA, "Bandit: Security linter for Python," v1.9.4, https://bandit.readthedocs.io/

[3] M. Brunsfeld et al., "Tree-sitter: An incremental parsing system for programming tools," https://tree-sitter.github.io/tree-sitter/

[4] OASIS, "Static Analysis Results Interchange Format (SARIF) Version 2.1.0," 2020.

[5] Astral Software, "Ruff: An extremely fast Python linter," rule ANN401. https://docs.astral.sh/ruff/rules/dynamically-typed-expression/

[6] Pylint, rule W0718 (broad-exception-caught). https://pylint.readthedocs.io/en/stable/user_guide/messages/warning/broad-exception-caught.html

---